

Week2Day1Assignment

Q/A — Claude AI Conversation

1. Prompt Q/A

You

Context: Act as a Test Automation Architect with Selenium Java and TestNG expertise. Instruction: - Ask me 5 questions ONE BY ONE before designing the framework - [MANDATORY] Ask: tool, language, design pattern, reporting, CI/CD - Ask the next question only after I answer - Framework should be Enterprise following the industry standard Persona: Senior Automation Architect Output: Folder/file structure with explanation Tone: [STRICTLY] Professional

Claude

Perfect — we'll build this like a real **enterprise-grade Selenium Java framework**, step by step.
I'll gather your requirements first, one question at a time.

Claude

Question 1 of 5

What type of application are you planning to automate?

- Web application
- API testing
- Both UI + API
- Mobile

Reply with your answer and I'll ask the next question.

You

Both UI and API — Banking domain (Internet Banking Portal)

Claude

Question 2 of 5

What programming language do you want to use with Selenium?

- Java (industry standard for Banking)
- Python
- C#

Reply with your answer and I'll ask the next question.

You

Java

Claude

Question 3 of 5

What design pattern should the framework follow?

1. Page Object Model (POM) — most common, easy to maintain
2. POM + Factory — enterprise standard
3. Screenplay Pattern — advanced

Reply with your answer and I'll ask the next question.

You

Page Object Model (POM) — Enterprise Standard

Claude

Question 4 of 5

What reporting tool and CI/CD integration do you need?

Reporting: Extent Reports / Allure / Both

CI/CD: Jenkins / GitHub Actions / Azure DevOps

Reply with your answer and I'll ask the next question.

You

Extent Reports + Jenkins

Claude

Question 5 of 5 (Final)

How do you want to manage test data, environments and configuration?

1. Simple — JSON/properties, minimal config
2. Moderate — .properties per environment, Excel test data
3. Advanced — External DB, secrets vault

Reply with your answer and I'll ask the next question.

You

Moderate — .properties file for each environment (dev/qa/prod), Excel for test data

2. Generated Framework Output

Claude

Perfect — now I have everything. Here is your **Enterprise Selenium Java Framework**.

Stack: Selenium + Java + TestNG + POM + Extent Reports + Jenkins

Recommended Folder Structure

```
project-root/  
src/  
main/java/  
pages/ -> Page Object classes (LoginPage.java, FundTransferPage.java)  
utils/ -> WebDriverUtils.java, WaitUtils.java, ExcelUtils.java  
base/ -> BaseTest.java (setup & teardown for all tests)  
listeners/ -> TestNG Listeners (screenshot on failure, Extent Reports)  
test/java/  
tests/ -> LoginTest.java, FundTransferTest.java  
resources/  
config/  
dev.properties -> Dev environment URL & credentials  
qa.properties -> QA environment config  
prod.properties -> Production config  
testdata/  
testdata.xlsx -> Excel test data (accounts, transfer amounts)  
reports/ -> Extent Reports HTML output  
testng.xml -> Test suite & parallel execution config  
pom.xml -> Maven dependencies  
Jenkinsfile -> CI/CD pipeline definition
```

Claude

Enterprise Best Practices applied:

- BaseTest handles all WebDriver setup and teardown centrally
- Listeners folder captures screenshots automatically on test failure
- Config properties file per environment — no hardcoded URLs
- Excel-driven test data keeps tests and data separate
- Jenkinsfile enables one-click CI/CD execution

3. Analysis / Review Comments

Claude

Test Strategy Document

Positive

Covered all important parts — scope, test types, environments, tools, and entry/exit criteria. Domain-specific output was generated using Chain-of-Thought prompting.

Negative

Test data strategy not described, no timeline provided, no approval or sign-off process included.

Improvements

Add a RACI chart, a test execution schedule, and a dedicated Test Data Management section.

Claude

Automation Framework Design

Positive

Good Page Object Model design, correct folder structure following industry standards, CI/CD pipeline via Jenkinsfile included.

Negative

No Log4j logger configured, no screenshot-on-failure utility in place, framework not set up for parallel test execution.

Improvements

Add a listeners/ folder with TestNG listeners to handle screenshots automatically. Add cross-browser configuration inside the properties file.

Claude

Overall: Both outputs were approximately 80% complete. The Chain-of-Thought approach produced domain-specific, well-structured results compared to a single one-shot prompt.